Tutorial for Virtual Screening of Peptides Using MolAICal and LightDock -- Also Applicable to Virtual Screening of Proteins and Nucleic Acids

Qifeng Bai
Email: molaical@yeah.net
Homepage: https://molaical.github.io or https://molaical.gitlab.io
School of Basic Medical Sciences
Lanzhou University
Lanzhou, Gansu 730000, P. R. China

1. Introduction

MolAICal can use the open-source LightDock[1] software (https://lightdock.org) and further enhance the docking accuracy of biomacromolecules through MM/GBSA or MM/PBSA methods. LightDock is an open-source (GPL-3.0 license), **docking framework** for **protein-protein**, **protein-peptide**, **protein-RNA,** and **protein-DNA** complexes. It handles flexible and challenging cases (e.g., transient interactions, low-affinity complexes, or systems requiring backbone/side-chain flexibility) **particularly well and serves as an extensible platform for testing new scoring functions, restraints**, or **optimization strategies.** LightDock adapts the Glowworm Swarm Optimization (GSO) algorithm[2], a bio-inspired swarm-intelligence method originally developed for multimodal function optimization. This tutorial uses the glucagon-like peptide-1 receptor (GLP-1R) and exendin-4, the first FDA-approved GLP-1R agonist, as demonstration examples (PDB ID: 7LLL.pdb).

This tutorial is only applicable for completion on the Linux operating system!

This tutorial demonstrates a computational workflow for peptide virtual screening through the integrated use of MolAICal and LightDock platforms. MolAICal enables efficient generation and 3D structure prediction of diverse peptide candidates, **including linear and cyclic variants with various secondary structures**, while LightDock employs advanced swarm optimization algorithms for accurate protein-peptide docking simulations. The combined approach allows researchers to screen thousands of peptide variants against target proteins, identifying promising binders based on binding energy and interaction patterns. By leveraging multi-scale modeling approaches and parallel computing capabilities, this workflow significantly accelerates the discovery process for therapeutic peptides, as well as protein inhibitors and nucleic acid binders, by replacing the peptides with proteins and nucleic acids. For more details, please see the tutorials of Protein-DNA and Protein-RNA docking, as well as the MolAICal manual.

2. Materials

2.1. Software requirement

1) MolAICal: https://molaical.github.io
2) NAMD (**CPU version**): https://www.ks.uiuc.edu/Research/namd/
(**Notice:** This tutorial can be run by **NAMD 2.x, 3.x, or a higher version.** For example, the command "**namd3**" substitutes for the command "**namd2**" in this tutorial **if you use NAMD 3.x version**. For a higher version of NAMD, users can use a similar replacement way as the previous example.)
3) PyMol: https://github.com/cgohlke/pymol-open-source-wheels ,
https://github.com/maabuu/pymol-wheels ,
https://pypi.org/project/pymol-open-source-whl ,
https://github.com/cnpem/PyMOL4Win

4) VMD: https://www.ks.uiuc.edu/Research/vmd

2.2. Example files

1) All the necessary tutorial files are downloaded from:
https://github.com/MolAICal/tutorials/tree/master/028-peptide_vs_lm

3. Procedure

3.1 Install external programs inside the MolAICal container (if finished, skipping step 3.1)

To address the library dependency issues in Linux, the Linux version of MolAICal (**Not for Windows version**) adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

To prevent the loss of installed programs (e.g., VMD and NAMD) when rebuilding the MolAICal image, refer to step 3 in **Appendix 2**.

✧ **Remember to use the "exit" command to quit the MolAICal virtual environment and return to the local computer for computation (primarily to avoid file-copying steps; execution within the container is also possible but requires copying data from the local computer to the MolAICal container).**

```
#> exit
```

Open the tutorial material folder "028-peptide_vs_lm":

#> cd 028-peptide_vs_lm

It will see the list of files:

- **7LLL.pdb:** the complex structure file in pdb format, which is from https://www.rcsb.org
- **dock_u.sh:** docking unit (Applicable to membrane protein receptors similar to this tutorial)
- **dock_u_nomem.sh:** docking unit (Applicable to non-membrane protein receptors)
- **dock_u_cyc.sh:** docking unit for cyclic peptide and membrane protein
- **dock_u_cyc_nomem.sh:** docking unit for cyclic peptide and non-membrane
- **mempro.pdb:** membrane receptor for this receptor. Preparation methods can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial.
- **new.pdb:** temporary file in the process of receptor construction.
- **P_peptide.pdb:** peptide ligand from the "7LLL.pdb". Preparation methods can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial.
- **restraints.list:** a file specifying active sites can reduce docking time for conformational search. Preparation methods can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial.
- **R_protein.pdb:** receptor file from the "7LLL.pdb". Preparation methods can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial.
- **R_P_complex.pdb:** the complex file containing the peptide ligand and receptor. Preparation methods can be found in Appendix 1 or the MolAICal protein-peptide docking tutorial.

Option: The DFIRE scoring function in LightDock requires standard amino acid residues and their atom names; thus, it is essential to inspect and repair the receptor or ligand before performing docking with the DFIRE scoring function. The repair command is as follows:

```
#> molaical.exe -call run -c sfile -i lmfix_rec.py -i protein.pdb -o protein_fixed.pdb
```

Here, the PDB file is OK and does not need to be dealt with the above option.

3.2. Peptide Generation

LightDock depends on the amino acid types supported by the scoring function. For example, DFIRE only supports the standard amino acid and residue name "MMB"; so it must use the argument "-nc" to avoid to add caps in the N- and C- terminal of the generated peptides because DFIRE cannot recognize them.

1) Generate the linear peptides by MolAICal

```
#> molaical.exe -call run -c pepgen -i -l 8 -n 2 -3d -nc -o linear_seq.txt -dir linear
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-i :** Call the external program. It includes all command-line arguments of external programs after '-i'.

- **-c (before -i):** It will generate peptides if its value is "pepgen".
- **-l:** Number of amino acid residues in the peptide
- **-n:** Number of unique sequences to generate
- **-3d:** Generate 3D structures using pyPept
- **-nc:** Do not add ac/am caps to sequences
- **-o:** Output file for generated sequences
- **-dir:** Output directory for 3D structures

For more details, users can refer to the MolAICal manual.

It will see the generated 3D peptides in the folder "linear" which can be used for further virtual screening.

2) Generate the linear peptides by MolAICal with fixed residues, which can be used for peptide modification.

#> molaical.exe -call run -c pepgen -i -l 8 -n 3 -rc 3 -nc -3d -dir mpep -s EEYVPIST -d none -fix 1 7

- **-dir:** Output directory for 3D structures
- **-rc:** Number of RDKit random conformers to generate (default: 1), which can give the wanted number of peptides.
- **-d:** Direction for sequence extension from seed [forward, backward, both, none].
- **-s:** Starting sequence (optional)
- **-fix:** Positions (1-indexed) to keep fixed during generation

For more details, users can refer to the MolAICal manual.

3) Generate the cyclic peptides by MolAICal. MolAICal also supports the peptide generation with disulfide bonds among different residues, etc (see MolAICal manual).

#> molaical.exe -call run -c pepgen -i -l 8 -n 3 -rc 3 -cc -3d -c 4 -dir cyclic

- **-cc:** Generate cyclic peptides (BILN/HELM only)
- **-c (after -i):** Number of CPU cores for parallel processing

For more details, users can refer to the MolAICal manual.

In the directories **"linear"**, **"mpep"**, and **"cyclic"**, three types of peptides are included for different purposes and applications. **These peptides will be used for the subsequent virtual screening.** Due to the **time-consuming** nature of peptide virtual screening, **this tutorial only extracts a limited number of peptides for demonstration**. Users can generate sufficient peptides based on their computational resources for research purposes.

3.3. Peptide Virtual Screening

MolAICal's virtual screening approach involves: creating a docking unit formatted as a MolAICal script file containing multiple tasks; **each unit docks a single molecule** (see Figure 1). Building on this, multi-core parallel processing screens multiple molecules concurrently to achieve virtual screening.

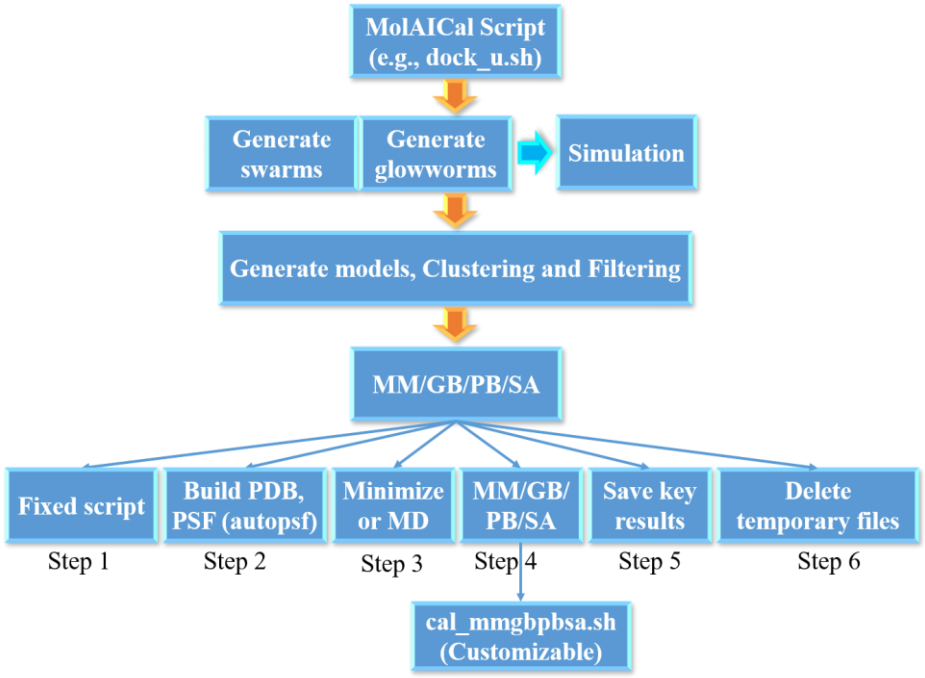

Figure 1 Docking unit.

Peptides in the **"linear"** and **"mpep"** folders can be screened using the same procedures. For **cyclic peptides** in the **"cyclic"** folder, **minor modifications are required**: During the MM/GB/PB/SA step, terminal residues must be patched with 'LINK'. This method also applies to disulfide bonds and similar modifications in peptide residues.

1) For linear peptides in the **"linear"** and **"mpep"** folders, run below command:

```
#> molaical.exe -call run -c sfile -i vs_ml.sh 1::=linear 2::=mempro.pdb 3::=R 4::=Z 5::=8 6::=1 9::=dock_u.sh
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **'::='**: Here, it is used to assign values to parameters in the specified order. Variables must immediately follow "::=" without any space.
- **'-c'**: It will run the scripts of MolAICal if its value is "sfile".
- **'1::='**: Work directory for storing molecules (e.g., peptides) to be screened. The default value is **'linear'**.
- **'2::='**: Receptor file. The default value is **'mempro.pdb'**.
- **'3::='**: Chain name of the first molecule (Corresponding: receptor file). The default value is **'R'**.
- **'4::='**: Chain name of the second molecule (Corresponding: ligand file). The default value is **'Z'**.
- **'5::='**: The number of CPU cores for the subtask (each docking job). The default value is **2**.

- **'6::='**: The number of CPU cores for the entire virtual screening job. The default value is **1**. This parameter determines the number of parallel pipelines to start. **Multiple parallel pipelines are highly memory-intensive. It is recommended to set this value to 1,** meaning only one pipeline will be activated. Unless available memory is very abundant, consider setting a higher number of pipelines. To further improve computational efficiency, the number of parallel cores can be increased for subtasks within the pipeline, for example: 5::=8.
- **'8::=':** Debug mode. Its value is 0 or 1. If its value is 1, it will do a test for the job to check the errors. If its value is 0, it will not do the debug and directly run the whole job. The default value is 0.
- **'9::='**: The docking script file in the MolAICal script format. The default value is **'dock_unit.sh'**.
- **'11::='**: The directory for output results. The default folder is **'dresults'**.
- For more details, please see the MolAICal manual.

2) For cyclic in the **"cyclic"** folders, run below command:

```
#> molaical.exe -call run -c sfile -i vs_ml.sh 1::=cyclic 2::=mempro.pdb 3::=R 4::=Z 5::=8 6::=1 8::=0 9::=dock_u_cyc.sh 11::=cyc_result
```

3.4. Ranking results

Here, the results in the folder "cyc_result" are chosen as the case. Users can refer to this case for their research.

1) Process only filtered candidates with a custom percentage (here it is 20%), please run the below commands:

```
#> molaical.exe -call run -c sfile -i rank_ppn.py cyc_result -m filtered -p 20 -fo filtered_results
```

Or

2) Process only MMGBPBSA with custom directory of output results:

```
#> molaical.exe -call run -c sfile -i rank_ppn.py cyc_result -m mmgbpbsa -mo mmgbpbsa_results -p 20
```

Or

3) Process both filtered and MMGBPBSA with default settings, and get the top 20% results:

```
#> molaical.exe -call run -c sfile -i rank_ppn.py cyc_result -p 20 -fo filtered_results -mo mmgbpbsa_results
```

- **-m {both,filtered,mmgbpbsa}:** Processing mode (default: both)
- **-p:** Percentage of top molecules to extract (default: 5.0)
- **-fo:** Filtered results output directory (default: vs_filtered_results)
- **-mo:** MMGBPBSA results output directory (default: vs_mmgbpbsa_results)
- "cyc_result" is the input directory that contains candidate peptide molecules for virtual screening.
- For more help information, please check the MolAICal manual

Task Resumption: If the task is interrupted, run the below command for task Resume:

```
#> molaical.exe -call run -c sfile -i vs_ml.sh 1::=linear 5::=8 7::=1 12::=continuevs
```

- **'7::='**: Check whether the docking job is completed. Its value is 0 or 1. If its value is 1, it will check the results in the MM/GB/PB/SA step. If its value is 0, it will check the results in the first virtual screening step. The default value is 1.
- **'12::='**: The more options for the docking script file in **'9::='**. The default value is an empty string, and the empty string is represented as \"\". If this input contains multiple parameters, separate them using the comma character ','. MolAICal will automatically convert each comma ',' to a space " ". It has some special values:
  - If **"createdir"**, it only creates the folders for virtual screening;
  - If **"startvs"**, it only runs virtual screening jobs;
  - If **"continuevs"**, it will handle the remaining unfinished virtual screening tasks;
  - If **"sn_{num}"**, it will set the ${num} of **parallel pipelines** which corresponds to **'6::='**;
  - Otherwise, run all jobs.

The PDB files completed with the virtual screening in the designated folder ${work_dir} (e.g., the folder "linear"), will be moved to the directory "${work_dir}_complete_folder". Then, directly running the previous virtual screening program will resume the task from the ${work_dir} folder, effectively achieving task resumption.

References

1. Jiménez-García, B.; Roel-Touris, J.; Romero-Durana, M.; Vidal, M.; Jiménez-González, D.; Fernández-Recio, J., LightDock: a new multi-scale approach to protein-protein docking. *Bioinformatics* **2018,** *34* (1), 49-55.
2. Krishnanand, K. N.; Ghose, D., Glowworm swarm optimization for simultaneous capture of multiple local optima of multimodal functions. *Swarm Intelligence* **2009,** *3* (2), 87-124.

Appendix 1

The Appendix 1 below **describes** the method for **preparing the materials for this tutorial**, which follows the **same** procedure as **protein-peptide molecular docking**; if you are already familiar with the protein-peptide molecular docking workflow, this section can be skipped.

1. Deal with molecular files

Load PDB file 7LLL.pdb into PyMol, and use the steps of **Figure a1** to save "R_P_complex.pdb", "R_protein.pdb", and "P_peptide.pdb", respectively.

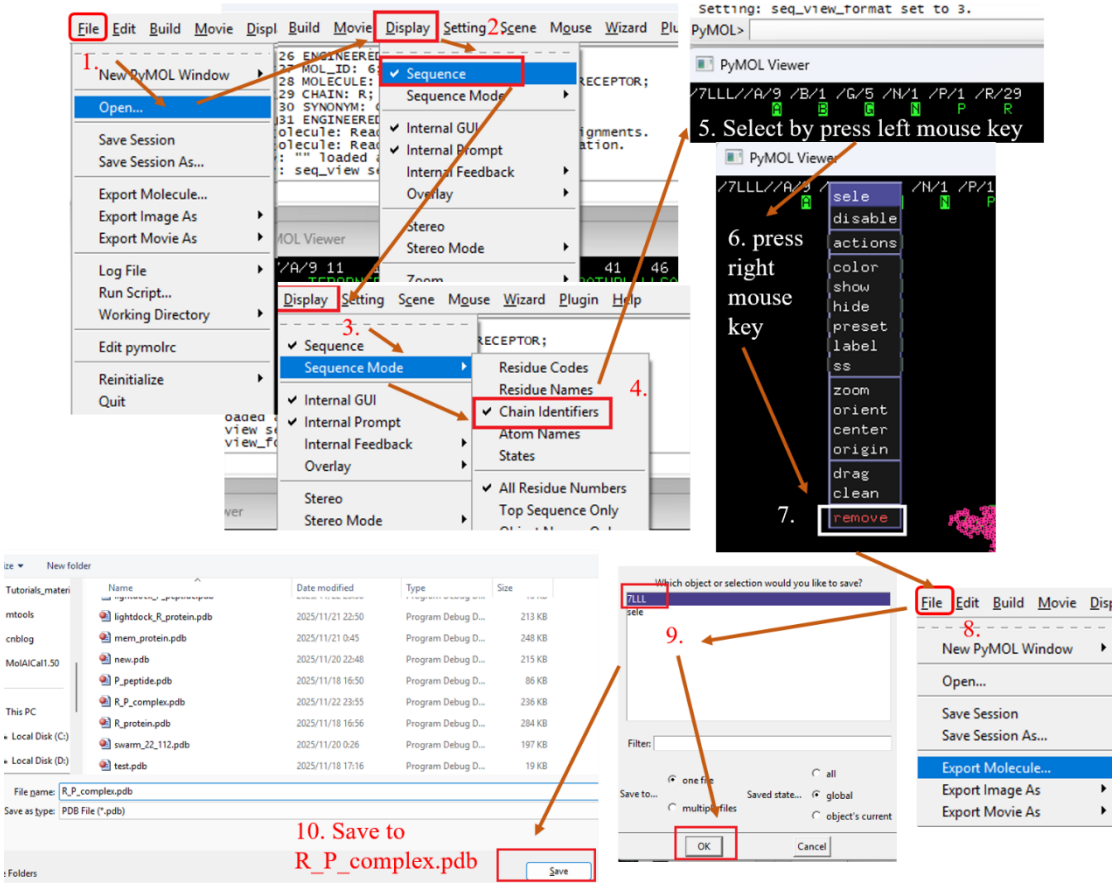

Figure a1

The files "R_P_complex.pdb" (chain name is "P" and "R"), "R_protein.pdb" (chain name is "R"), and "P_peptide.pdb" (chain name is "P") are extracted from the complex of the glucagon-like peptide-1 receptor (GLP-1R) and exendin-4, the first FDA-approved GLP-1R agonist (PDB ID: 7LLL.pdb). The agonist exendin-4 is a peptide. If users want to **perform protein-protein or protein-nucleic acids virtual screening**, please **substitute the peptide file with the protein or nucleic file** because the peptide can be considered a small protein.

Options: The default chain name of generation peptides is "Z" in MolAICal. **If the chains of two different molecules are given the same name** (e.g., both named Chain 'Z'), in such cases, users can rename the receptor (e.g., Chain A but not Chain Z) by calling VMD through MolAICal. **If this situation does not occur, please skip** this step and the following commands. The command is as follows:

```
#> molaical.exe -call run -c vmdargs -i -pdb R_protein.pdb -args none set sel [atomselect top all], \\\$sel set chain A, \\\$sel writepdb R_protein_A.pdb
```

Note:

- It includes **all command-line arguments** of **external programs**, but the **'-i'** parameter must be the last one specified, while all other identifiers (e.g., '-call', '-c', etc.) must be **before '-i'.**
- **'-pdb'** means to load pdb file into VMD.
- **-c:** It will run the VMD command once in the Tk Console if its value is "vmdargs".
- The Strings after **'-args'** are represented to the commands which can be run on the Tk console of VMD.
  - The first parameter after **'-args'** can be a package name of VMD or **'none'**. If the first parameter is **'none'** after **'-args'**, it will omit the package for loading. For example, it will run the command "package require **autopsf**" in the Tk console of VMD if the first parameter is **'autopsf'** after **'-args'**. It will not run the command "package" if the first parameter is **'none'** after **'-args'**.
  - The string following **'-args'** contains one command that can be executed in VMD's Tk console before each comma **','**; in other words, the comma character replaces the process of entering each command in VMD's Tk console and pressing the Enter key. This allows multiple lines of commands to be executed with just a single command.
  - Some special characters, such as the character **"$"**, require the use of the escape character **"\"**. For special characters, MolAICal uses three escape characters **"\"** (i.e., **"\\\"**).

In this tutorial, the above option is not used; the file "R_protein.pdb" (chain name is "R") is used because they have different chain names.

2. Make residue restraint file

Creating a restraint file for residues is like identifying the key residue sites in the active pocket, around which the molecule will be docked. **Users can customize these key residues based on literature introductions or directly from experience**.

The residue representation format in the restraint file is:

```
Chain.Residue_Name.Residue_Number[.Residue_Insertion]
```

Note: Here, Residue_Insertion is an optional single-character identifier appended to the standard residue number, commonly used in PDB files to distinguish inserted residues without renumbering the entire sequence. In bioinformatics and structural biology, insertion codes are critical for accurately tracking and describing specific protein residues, particularly when dealing with sequence variations or comparisons.

When Residue_Insertion is omitted, the general format is:

```
Chain.Residue_Name.Residue_Number
```

Or

```
Chain.Residue_Name.Residue_Number P
```

Note: The label "P" indicates that this restraint is passive (as opposed to active restraints, which have no label). **Passive residue restraints are not recommended unless their implications are fully understood.**

Here, MolAICal is used to generate the residues list file (named 'rec.dat') of the receptor (protein) within 4.0 Å of the ligand (peptide) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args R P 4.0 rec.dat R
```

Note: the terms 1st, 2nd, 3rd, and 4th below represent the first argument, the second argument, the third argument, and the fourth argument, respectively, and so on.

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'get_res.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** The chain name of the selected molecule;
- **2nd:** The chain name of reference molecule used for selection;
- **3rd:** The residues of the selected molecule within the specified distance from the reference molecule will be selected;
- **4th:** The saved filename;
- **5th:** Specifies the molecular type, 'R' stands for Receptor and 'L' stands for Ligand.

Similarly, MolAICal is used to generate the residues list file (named 'lig.dat') of the ligand (peptide) within 4.0 Å of the receptor (protein) as key residues, as shown below:

```
#> molaical.exe -call run -c sfile -i get_res.tcl -pdb 7LLL.pdb -args P R 4.0 lig.dat L
```

Merge the 'rec.dat' and 'lig.dat' into the restraint file named 'restraints.list' as follows:

```
#> molaical.exe -call run -c ecommand -i cat rec.dat lig.dat > restraints.list
```

In many cases, the specific details of the ligand are unknown. Therefore, this tutorial only uses the key residues of the receptor (the first molecule) for constraints (and manually removes unnecessary amino acids based on experience), with the commands as follows:

```
#> molaical.exe -call run -c ecommand -i cp rec.dat restraints.list
```

3. Setup for molecular docking

Firstly, run the setup for swarm generation as follows:

```
#> molaical.exe -call run -c ldock -i lightdock3_setup.py R_protein.pdb P_peptide.pdb --noxt --noh --now -sp -rst restraints.list
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run molecular docking (protein-protein, protein-peptide, protein-DNA or protein-RNA) by calling "LightDock" if its value is "ldock".
- The 1st and 2nd arguments after "lightdock3_setup.py" are receptor and ligand files, respectively.
- **--noxt**: If this option is enabled, LightDock ignores OXT atoms. This is useful for several scoring functions that don't understand this special type of atom.
- **--noh**: If this option is enabled, LightDock ignores hydrogen atoms. This is relevant for dfire, fastdfire or dfire2 scoring functions (among others). Only removes atoms starting with H character, thus not recognizing types such as 1H, etc.
- **--now**: If this option is enabled, crystal waters will be removed.
- **--anm:** If this option is enabled, the ANM mode is activated and backbone flexibility is modeled using ANM (via ProDy). **This parameter may cause structural distortion** in docking results by activating backbone flexibility modeled via ANM (using ProDy). **Avoid using it unless implementing better conformational sampling (e.g., energy minimization)**. Instead, pre-generate multiple conformations to bypass backbone flexibility during docking.
- **-membrane:** When enabled, this flag considers the receptor partner is aligned to the Z-axis and filters out swarms that would not be compatible with a transmembrane domain. To do so, it makes use of the residues provided as receptor restraints.
- **-transmembrane:** The argument "--transmembrane" has the opposite function of "--membrane". When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.
- **-sp:** If enabled (False by default), writes in the init folder debugging extra files.
- **-r, -rst restraints_file:** If restraints_file is provided, residue restraints will be considered during the setup and the simulation. If restraints_file is not provided, the setup and the simulation will focus on the entire receptor.

✧ It will produce the folder named **'init',** which contains the molecules in PDB format that represent the glowworms of swarms for further docking.

✧ The files 'lightdock_R_protein.pdb' and 'lightdock_P_peptide.pdb' are re-generated from 'R_protein.pdb' and 'P_peptide.pdb' by LightDock

Open 'lightdock_R_protein.pdb' and all PDB files with the prefix "starting_positions*" in the folder named 'init' by PyMol (see Figure a2):

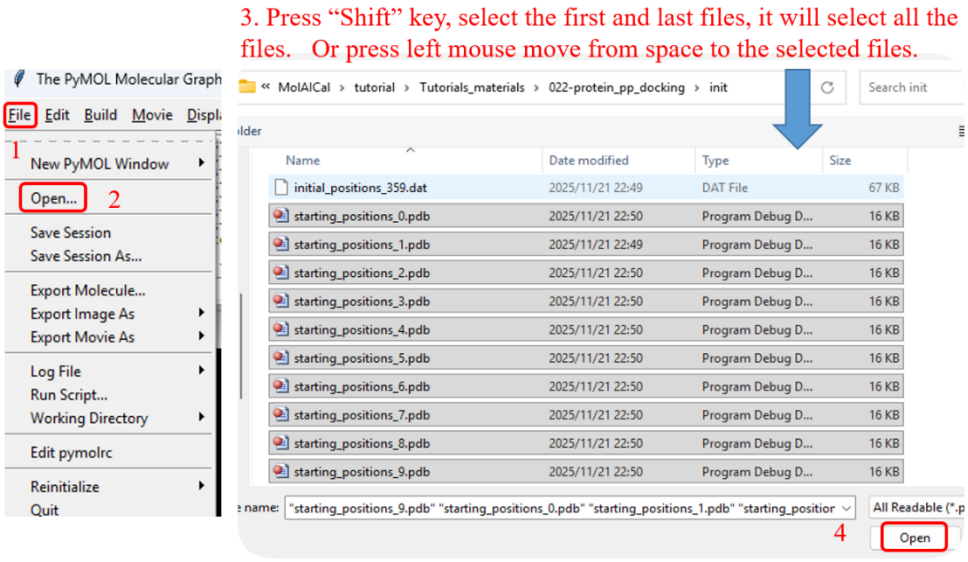

Figure a2

Please see the results in Figure a3. It is evident that many glowworms are **located in the intramembrane region**, while the ligand peptides are positioned in the extracellular-facing pocket at the center of the 7-transmembrane helices of the membrane protein (as shown in Figure a3). **These glowworms in the intramembrane region are not only unreasonable but also consume significant computational resources and time during calculations**, potentially compromising docking results.

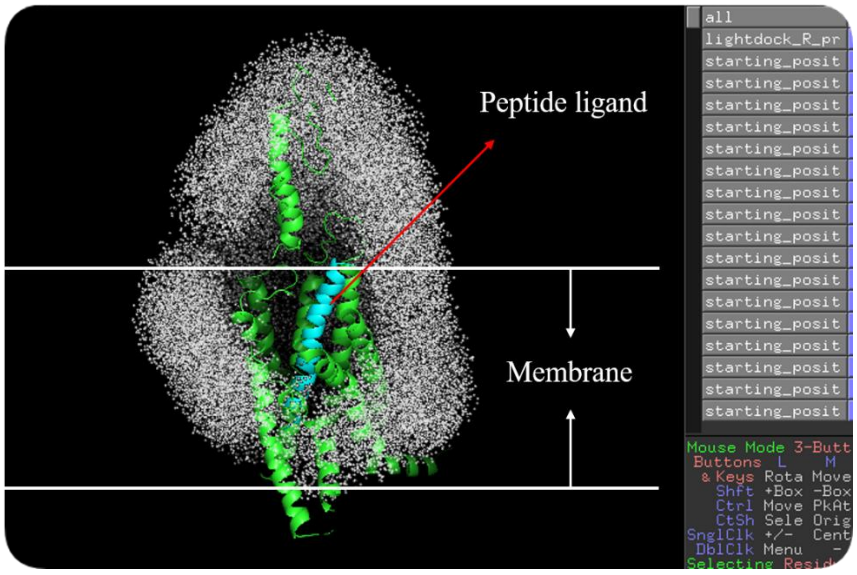

Figure a3

Therefore, MolAICal offers a simple membrane generation method to minimize the generation of unreasonable intramembrane glowworms:

3.1 (Option) Set the membrane in the receptor

Skip this step if the target receptor is not a membrane protein!

3.1.1 (Option) First, open "lightdock_R_protein.pdb" using VMD. Through VMD, it can visually inspect the receptor in x, y, and z directions. To determine whether the transmembrane region of the membrane protein (facing the extracellular side) is roughly aligned along the z-axis, if it is already aligned, no further action is needed. If it is not roughly aligned along the z-axis, use MolAICal to adjust its orientation to align with the z-axis. The command is as follows:

```
#> molaical.exe -call run -c sfile -i align_axis.tcl -pdb R_protein.pdb -args protein 2 new.pdb
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'align_axis.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to represent the blank space " " (e.g., protein,and,chain,B);
- **2nd:** If the protein aligns horizontally (lying flat), change 0 to 2;
- **3rd:** The output file name;
- **4th:** The selected axis for "transaxis" of VMD, which can be x, y, z, or "". "" can be equal to no character, including space for inputting.

The file "new.pdb" is along the z-axis, which will be used for the steps below.

3.1.2 Once, make sure the receptor is roughly along the z-axis. Identify two positions on the receptor for membrane generation.

- Firstly, open "R_protein.pdb" in VMD
- Secondly, hold down the numeric key '1', then roughly click with the mouse near the inner and outer edges of the membrane. These positions can be adjusted according to experimental requirements, primarily to restrict the generation of glowworms between the membranes.
- Obtain the z-axis values of these two positions and use them as the min and max values of the range [min, max] (as shown in Figure a4).

Here, [min, max]=[-29.0, 10.0]. These are just the rough numbers. Users can adjust them according to the research purpose. Then, run the command below for membrane generation:

```
#> molaical.exe -call run -c sfile -i gen_mem.tcl -pdb new.pdb -args protein -29.0 10.0 mempro.pdb
```

- **'-call'**: Interact with external programs or commands. Its value can be 'set' or 'run'. 'set' means to set the environment of external programs, including name and path. 'run' means to call the external programs, which can search the path of the program from the set environment file.
- **-c:** It will run the VMD scripts of MolAICal if its value is "sfile", here it calls the script file 'gen_mem.tcl'.
- **'-pdb'** means to load pdb file into VMD.
- **1st:** Atom selection (same as the command "atomselect" in the Tk console of VMD). If there is one more than a letter (e.g., protein and chain B), it will use the comma "," to represent the blank space " " (e.g., protein,and,chain,B);
- **2nd:** The minimum z-axis for membrane generation;

- **3rd:** The maximum z-axis for membrane generation;
- **4th:** The saved output file name;
- **5th:** The delta value is used to [($max - $delta), $max], which represents the range of values used to determine the maximum and minimum x,y values along the z-axis region. This ensures that membrane particles in the cross-section of the membrane are not located inside the membrane protein. Default value: 3.0;
- **6th:** Safety buffer for the bounding box (set 0 to strictly use protein limits). Default value: -2.0;
- **7th:** Membrane Padding (Angstroms). Default value: 15.0;
- **8th:** The minimum spacing between particles. Default value: 3.5;
- **9th:** The maximum spacing between particles. Default value: 6.0;
- **10th:** The distance to keep away from protein atoms (Angstroms). Default value: 4.0.

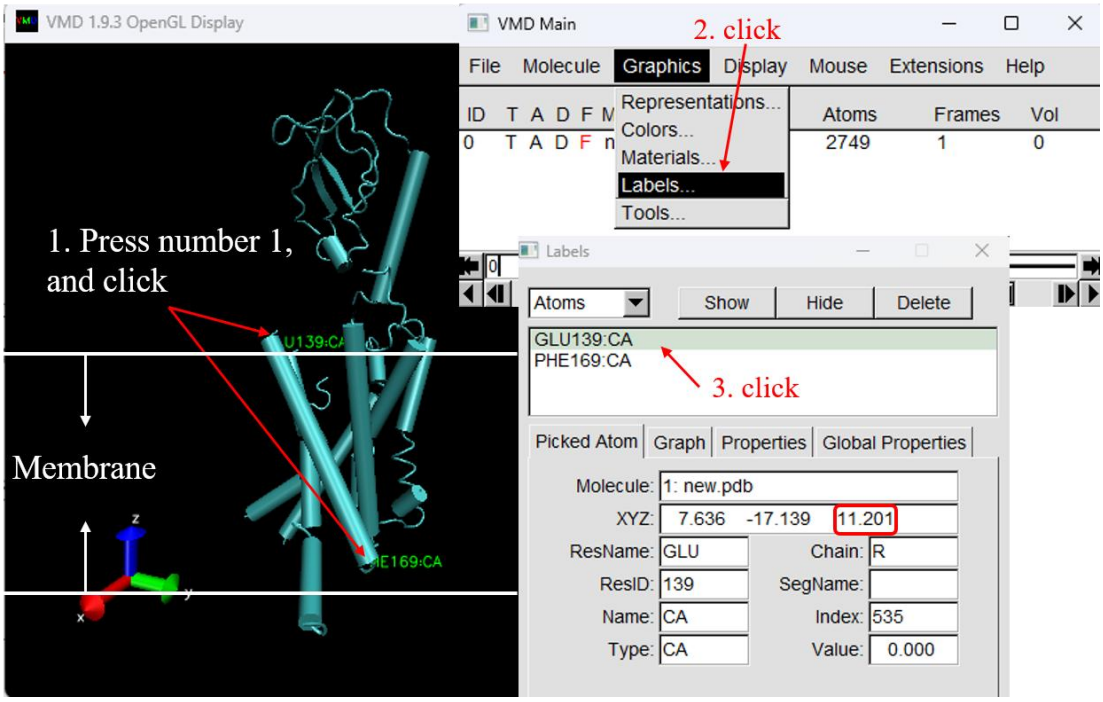

Figure a4

3.1.3 It will generate the file named "mempro.pdb", which contains receptor and membrane molecules.

- **Option:** Remove the previously generated files (If not deleted, it will cause some errors). If the previously generated files do not exist, skip this step.

```
#> molaical.exe molaical.exe -call run -c ecommand -i rm -rf init swarm_* lightdock_*
```

Re-run the setup for swarm generation as follows (add argument: --membrane):

```
#> molaical.exe -call run -c ldock -i lightdock3_setup.py mempro.pdb P_peptide.pdb -membrane --noxt --noh --now -sp -rst restraints.list
```

Notice: -transmembrane: The argument **"--transmembrane"** has the opposite function of **"--membrane".** When enabled, this flag considers the receptor partner to be aligned to the Z-axis and filters out swarms that would not be inside the membrane.

It is obvious that the number of generated swarms is smaller than before. Open '**lightdock_mempro.pdb**' and all **PDB** files with the **prefix** "starting_positions*" in the folder named **'init'** by PyMol as shown in Figure a2. **Regenerate** glowworms of swarms as shown in **Figure a5;** notably, no swarms are generated between membranes, which is reasonable.

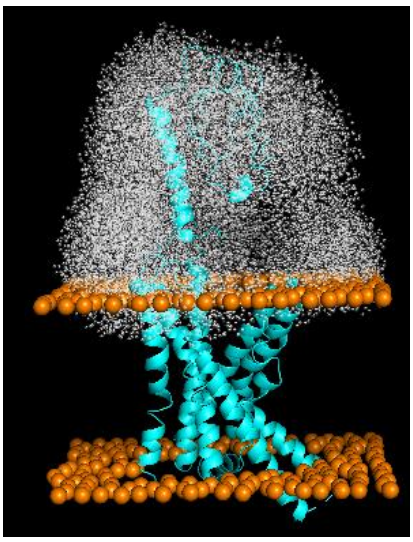

Figure a5

Now the swarms and the receptor are handled.

Appendix 2

Install external programs inside the MolAICal container (if finished, only for Linux Version MolAICal)

Please note that the configuration of MolAICal differs between the Windows and Linux versions: the Linux version utilizes the udocker container approach and requires setup within the container, whereas the Windows version does not.

To address the library dependency issues in Linux, the Linux version of MolAICal adopts a container-based approach. If **external programs need** to be invoked, it is **recommended** to install them **within** the MolAICal **container**. If **no external** programs are required, this step can be **ignored**. This tutorial requires the external programs NAMD and VMD, and the steps are as follows:

a) First of all, copy the files to the MolAICal container.

```
****** 1.  Go to the container file system, it will enter the "/root" ******
#> molaical.exe -eset shell in

****** 2.  The first part of 'cp' is from the local machine, and the second part is in the container; ******
****** The VMD and NAMD packages can be moved into the container by the command below ******
#> cp /home/user/<local machine file> /root/soft

****** 3.  Exit container file system ******
#> exit
```

b) Secondly, enter the container virtual environment of MolAICal:

```
#> molaical.exe -eset sys run molaical
```

Note: 'molaical' is the container name.

c) Installing software in the container virtual environment of MolAICal is the same as installing it on the host local machine. Here, we take the installation of VMD and NAMD as examples:

1) For NAMD:

Extract the NAMD files, assuming the extracted folder is named namdcpu. Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n NAMD -p "/root/soft/namdcpu/namd3"
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system. The string "VMD" and "NAMD" (case insensitive) after "-n" is fixed for the paths of VMD and NAMD. To ensure the reproducibility of MM/GBSA results, it is recommended to use the CPU version of NAMD, as the "seed" parameter appears to be ineffective for reproducibility in the CUDA version of NAMD.

2) For VMD:

Following the steps below:

✧ Extract the VMD files:

```
#> tar -xzvf vmd-xxx.tar.gz
```

Note: Please **replace** the above paths for VMD and NAMD with the **actual paths** on the users' system.

✧ Modify the install path in a file named "**configure**" in the decompressed folder of VMD:

```
The default: $install_bin_dir="/usr/local/bin"; modify it to →
$install_bin_dir="/root/soft/vmd193/bin";

The default: $install_library_dir="/usr/local/lib/$install_name"; modify it to →
$install_library_dir="/root/soft/vmd193/lib/$install_name";
```

✧ Install VMD:

```
#> cd vmd-xx
#> ./configure LINUXAMD64
#> cd src
#> make install
```

Note: Please run **./configure**, and select the right types for the used computers. Here, it is "LINUXAMD64".

✧ Then specify its path to MolAICal using the following command:

```
#> molaical.exe -call set -n VMD -p "/root/soft/vmd193/bin/vmd"
```

So far, all the installations and configurations of NAMD and VMD are complete in the MolAICal container.

✧ **Remember to use the "exit" command to quit the MolAICal virtual environment and return to the local computer for computation (primarily to avoid file-copying steps; execution within the container is also possible but requires copying data from the local computer to the MolAICal container).**

```
#> exit
```

3) Save and load the modified container (Option)

To preserve changes made within a container and avoid reinstalling software later, since all data will be lost if the container is deleted.

◆ The first step is to obtain the container ID and name:

```
#> molaical.exe -eset sys ps
```

◆ Secondly, clone this container as follows:

```
#> molaical.exe -eset sys export --clone -o molaicalv2.tar molaical
```

Note: 'molaical' is the container name. It can also be the container ID. If an error is reported, it may be due to permission issues with the mounted file system, which can be ignored.

◆ Thirdly, import the cloned container **if needing**:

```
#> molaical.exe -eset sys import --clone --name molaicalv2 molaicalv2.tar
```

◆ Now, change the loaded new container name to the default container name "molaical", the previous container is renamed to "bakmolaical", the commands are as follows:

```
#> molaical.exe -eset sys rename molaical bakmolaical
#>molaical.exe -eset sys rename molaicalv2 molaical
```

Notice: Please note that a **container** (**dynamic**) is generated from an **image** (**static**). Operations such as software installation must be performed within the dynamic container; if cloning this container, restoring it will still be based on the original underlying image.

For more information, please see the manual of MolAICal or run "molaical.exe -eset sys --help" to get more help details.